```
}

class OnlineShopOrder extends OrderFulfillment
{
    public function wholesaleOrder()
    {
        throw new Exception('wholesaleOrders not supported');
    }

    public function customerOrder()
    {
        // online customer stuff
    }
}
```

Here we've decided to create a new `OnlineShopOrder` class, using `OrderFulfillment` as its parent. `OnlineShopOrder` is obliged to implement `wholesaleOrder()` but has no use for it, so it replaces the parent implementation with an exception (it could, of course, silently fail or some other error state). What it does not do is behave at all in the same way as `OrderFulfillment::wholesaleOrder()` would be expected to (by adjusting inventory levels, sending messages, and so on), and we've violated the Liskov Substitution Principle. Exceptions being thrown on non-implemented methods is a classic code smell. Should you ever encounter them in the wild, it is a clear indication that application of the ISP is sorely needed somewhere in your codebase!

The examples above are contrived ISP violations writ small. It should not be too much of a cognitive leap to imagine the problem on a larger scale, where blob classes are offering interfaces that are difficult to document or understand (and use) or forcing subclasses to overload dozens of unnecessary public functions. One solution to our `OrderFulfillment` problem is, of course, to split it into specialized interfaces: one for shops and one for warehouses. An even better solution would be to simplify the interface to a single method and have `OrderFulfillment` composed with the correct components to service its client at run-time. We'll see some patterns for doing this later.

Dependency Inversion Principle (DIP)

Finally, there's dependency inversion. We've already met this principle at the end of the previous section. If you remember, we had a class called `StockTaker` with a dependency on inventory classes. Let's see what it looks like when we don't use dependency inversion.